Day 2: Variables, Types & DSA Complexity

Roadmap	AI Automation Engineering + Government Technical Career
Study Date	8 November 2026
Learning Duration	2 hours
Level	Beginner - building on Day 1

Day 2 Learning Outcome: You learned how variables refer to values, how Python handles types dynamically, how to convert between common data types, how `input()` works, and how names can refer to mutable or immutable objects. You also practised recognising common Big-O growth patterns.

Learning Objectives

- Understand variables and assignment in Python.
- Apply Python naming rules and use readable `snake_case` names.
- Recognise dynamic typing and inspect values with `type()`.
- Convert values using `int()`, `float()`, `str()`, and `bool()`.
- Understand why `input()` returns a string.
- Explain object references, `id()`, and mutable versus immutable objects.
- Identify O(1), O(n), O(n^2) and O(log n) patterns.

1. What is a variable?

A variable is a name that refers to an object (a value) in a Python program. You create or assign a variable using the equals sign (`=`).

```
student_name = "Suraj"
age = 24
cgpa = 8.02
is_learning = True

print(student_name, age)
# Output: Suraj 24
```

> **Important distinction:** In Python, a variable is best understood as a name bound to an object, rather than as a fixed box that permanently stores one particular type of value. A name can later be assigned to a different object.

2. Variable Naming Rules

- A name may contain letters, digits, and underscores (`_`).
- A name must not begin with a digit.
- Spaces are not allowed in variable names.
- Names are case-sensitive: `age`, `Age`, and `AGE` are different.
- Python keywords such as `if`, `for`, `class`, and `True` cannot be used.

Name	Valid?	Reason
`student_name`	Yes	Letters and underscore; descriptive.
`marks2`	Yes	A digit is allowed after the first character.
`_total`	Yes	An underscore may start a name.
`2marks`	No	A name cannot start with a digit.
`student name`	No	Spaces are not allowed.
`class`	No	`class` is a Python keyword.

Python naming convention: snake_case
For ordinary variable and function names, use lowercase words separated by underscores (e.g., `student_name = "Asha"`, `monthly_income = 25000`).

3. Dynamic Typing & Checking Types

Python is dynamically typed. You do not normally declare a variable's type in advance. Use `type()` to inspect the type of a value or object.

```
value = 100
print(type(value))
# Output: <class 'int'>

value = "Python"
print(type(value))


d = True
print(type(d))

# Output: <class 'str'># Output: <class 'bool'>
```

4. Type Conversion (Type Casting)

Type conversion means changing a value from one data type to another.

```
# int() - convert to an integer
age_text = "24"
age = int(age_text)
print(age + 1)
# Output: 25

# float() - convert to a floating-point number
price_text = "99.50"
price = float(price_text)
print(price + 0.50)
# Output: 100.0

# str() - convert to a string
marks = 85
message = "Marks: " + str(marks)
print(message)
# Output: Marks: 85

# bool() - convert to a Boolean
print(bool(1))       # Output: True
print(bool(0))       # Output: False
print(bool(""))      # Output: False
print(bool("Python")) # Output: True
```

5. Getting Input from the User

The `input()` function displays a prompt and waits for the user to type something. It **always returns a string**.

```
name = input("Enter your name: ")
print("Hello,", name)
# Output (if user types 'Suraj'): Hello, Suraj

age = int(input("Enter your age: "))
print("Next year you will be", age + 1)
# Output (if user types '24'): Next year you will be 25
```

6. Object References, id(), and Mutability

A variable name refers to an object. The built-in `id()` function returns an identity value for an object.

```
a = 50
b = a
print(a is b)
# Output: True
```

Mutable vs Immutable Objects:

- **Mutable** (e.g., `list`, `dict`, `set`): Can be changed after creation.
- **Immutable** (e.g., `int`, `float`, `bool`, `str`, `tuple`): Cannot be changed after creation. Reassigning variables creates a new object.

```
# Mutable Example
first = [1, 2]
second = first
second.append(3)
print(first)
# Output: [1, 2, 3] (Both names refer to the same modified list)

# Immutable Example
language = "Python"
language = language + " Programming"
print(language)
# Output: Python Programming (A new string object is created)
```

7. DSA: Understanding Big-O Patterns

Pattern	Complexity	Reason
One fixed operation	O(1)	Work does not grow with n.
Repeatedly halve search range	O(log n)	Problem size is divided by 2 each step.
One loop from 0 to n-1	O(n)	Approximately n iterations.
Two nested loops	$O(n^2)$	Approximately n × n iterations.

8. Big-O Code Examples

```
# Example A: One Loop - O(n)
n = 3
for i in range(n):
    print(i, end=" ")
# Output: 0 1 2

# Example B: Nested Loops - O(n^2)
n = 2
for i in range(n):
    for j in range(n):
        print(f"({i},{j})", end=" ")
# Output: (0,0) (0,1) (1,0) (1,1)

# Example C: Consecutive Loops - O(n)
# O(n) + O(n) + O(n) = O(3n) which simplifies to O(n)
for i in range(n): pass
for j in range(n): pass
for k in range(n): pass
```

9. Common Mistakes to Avoid

- Using a number as the first character: Use `student2`, not `2student`.
- Assuming `input()` returns a number: Convert it with `int()` or `float()` before arithmetic.
- Adding complexities for consecutive loops incorrectly: O(n) + O(n) = O(2n) = O(n).
- Thinking immutable means a variable cannot be reassigned: The *object* cannot be changed; the name can still refer to another object.

10. Completion Record & Next Steps

Assignments	6 / 6 completed
Assessment	10 / 10
Difficulty reported	Moderate
Independent coding	Confirmed: able to write programs without copying
Revision focus	Time complexity and Big-O patterns

Study method: Read a concept, close the notes, explain it aloud, write a small program from memory, then run it and check the result.

Next lesson preview: Day 3 covers Python data types and type conversion in greater depth, along with an introduction to arrays/lists in DSA.